System Design Notes: Consistency

Nov 2, 2026 · @Salike Hassan

Eventual consistency

Does it only belong to distributed systems? Mostly yes. It shows up as soon as the same data is kept in more than one place: read replicas, caches, or separate databases behind separate services. A database with a single copy doesn't have this problem.

Definition: if writes stop, all copies eventually agree. When an update happens, the other replicas catch up to the same data after some latency.

Example: Priya changes her Instagram username from "Priya" to "Priya Shah".

- The write goes to the **leader** (the primary).
- The **followers** (read replicas) copy it from the leader a moment later.
- A read that lands on a follower in that moment still returns "Priya".
- Once replication catches up, every copy says "Priya Shah".

In any given second, most reads return the fresh value and a few return a stale one. That is the whole trade: we accept a short delay in exchange for cheap, fast, highly available reads.

Eventual consistency alone is too weak for some cases, so the next rungs add guarantees on top: **read your writes**, **monotonic reads**, **causal**, and at the top **linearizable**.

Read your writes

The guarantee: you always see your own updates. Priya changes her name and then reloads her profile. If that read lands on a follower that hasn't caught up, she sees the old name and thinks the save failed.

This is about the **person who wrote**. Two ways to fix it:

1. **A time window.** For 60 s after a write, read that user's data from the leader. Simple. The window just has to be longer than your usual replication lag (30 s or 60 s, tune it).
2. **A version token (better).** Every write gets a version, say v42, and the write returns it. Priya's reads carry "need at least v42". If a replica is on v41, it isn't ready yet, so we **skip it** and read from another replica or the leader. It costs a little latency only when we

have to skip.

How do we know a value is the latest?

A value can't tell you it's the latest. So we compare a number that only goes up:

- Every write gets an **increasing version**: a row version, a log sequence number (LSN) or a commit timestamp.
- The write **returns that number** to the client. That is the token.
- Each replica knows **how far it has applied**, its applied version.
- A read is good enough if the replica's applied version is **at least the token**.

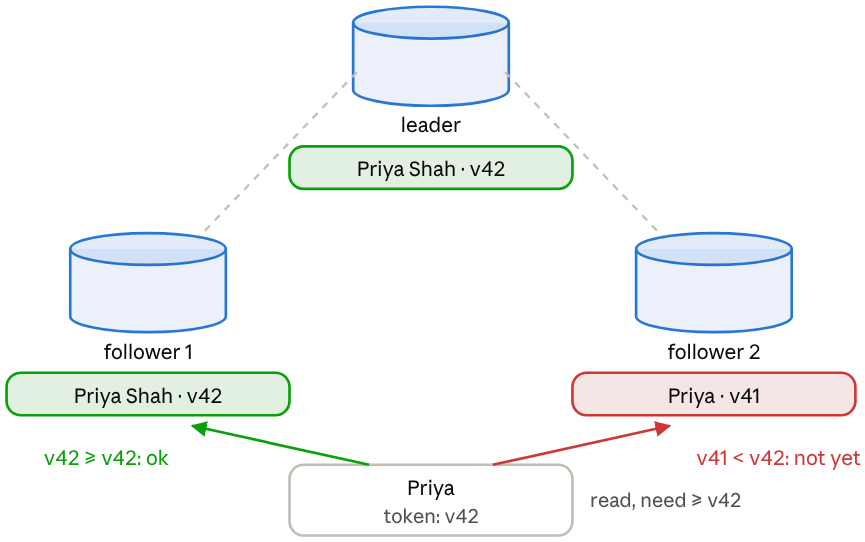
That answers "latest *for me*", meaning my own writes. For "latest *for anyone*", the only sure way is to ask the leader or a majority, which is **linearizable** (below).

How real databases do it

From my own knowledge, not your slides, so please check against your versions.

System	How it handles it
PostgreSQL	Compare `pg_current_wal_lsn()` on the primary with `pg_last_wal_replay_lsn()` on the replica. The LSN is the version.
SQL Server / Azure SQL	Readable secondaries in an availability group can lag. Use the primary for reads that must be fresh. Lag shows in `sys.dm_hadr_database_replica_states`.
Azure Cosmos DB	Five levels: strong, bounded staleness, session, consistent prefix, eventual. **Session** gives read your writes and monotonic reads within a session, using a session token, which is the version token above.
DynamoDB	Reads are eventually consistent by default. `ConsistentRead = true` reads from the leader for the latest value.

Read your writes: the version token says which replicas are good enough



Fix 1: for 60 s after a write, read from the leader.
Fix 2: the write returns a version token. Skip any replica below it (use another replica, or the leader).

read your writes: window and version token

Monotonic reads

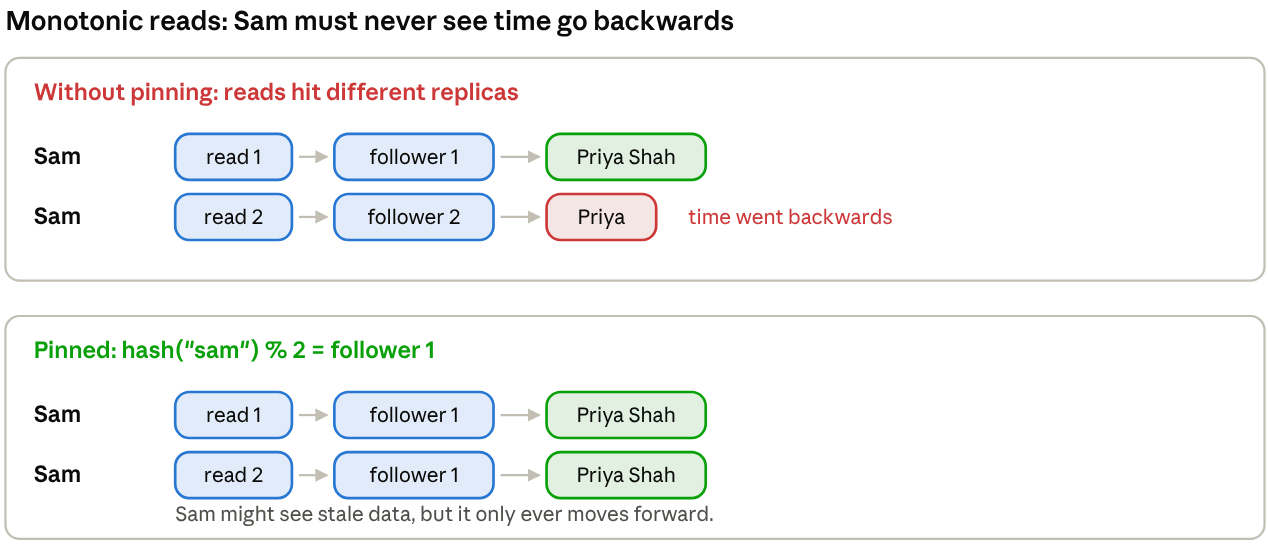
Read your writes protects the person who wrote. What about everyone else? Say Sam views Priya's profile.

- **Read 1** lands on follower 1, which has caught up: "Priya Shah".
- Sam reloads. **Read 2** lands on follower 2, which hasn't: "Priya".

Time went backwards. **The guarantee:** never see an older value after a newer one.

The simple fix: pin each user to one replica, for example `hash("sam") % 2` picks follower 1 every time. Sam might see stale data, but what he sees only ever moves forward.

If the pinned replica fails, move Sam to another one that is at or above the last version he saw (the same version number idea as above).



monotonic reads: unpinned vs pinned

Causal consistency

Take a post with comments, and replies to those comments. **The guarantee: if B depends on A, everyone sees A first.** A reply must never show up before the comment it answers.

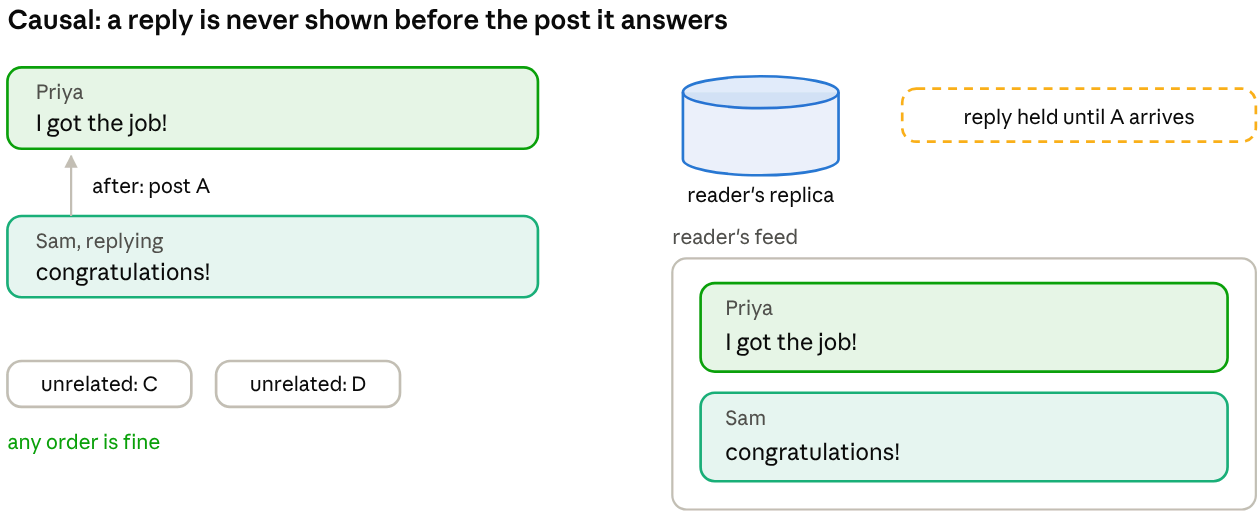
Writes that are **unrelated** (C and D) can still show up in any order. That freedom is exactly what keeps causal consistency cheap. We only pay for ordering where there is a real dependency.

How do we keep the sequence?

Record the dependency when the write happens, and enforce it when the data becomes visible:

1. **On write,** a reply carries what it depends on: its `parent_id` ("after: post A"), or the version it had seen.
2. A replica that receives the reply **before** the parent **holds it back** until the parent has arrived, then makes it visible.
3. **On read,** a reply is returned only if its parent is visible. So a thread always loads parent first.
4. Among siblings with no dependency, any simple tie-breaker works, such as created time or id.

For comments, that means a table like `comments(id, post_id, parent_id, created_at)`. A single leader's replication log already keeps its own writes in order. Across leaders or partitions, systems track the dependencies explicitly, with version vectors, dependency lists or hybrid logical clocks.



causal: the reply waits for the post

Linearizability (strong)

The guarantee: the system behaves as if there were only **one copy** of the data, and every operation happens at a **single instant**. Once any read has returned the new value, every later read, by anyone, must return it too.

In practice that means a **leader, or a majority of replicas, decides** whether a write is accepted or rejected.

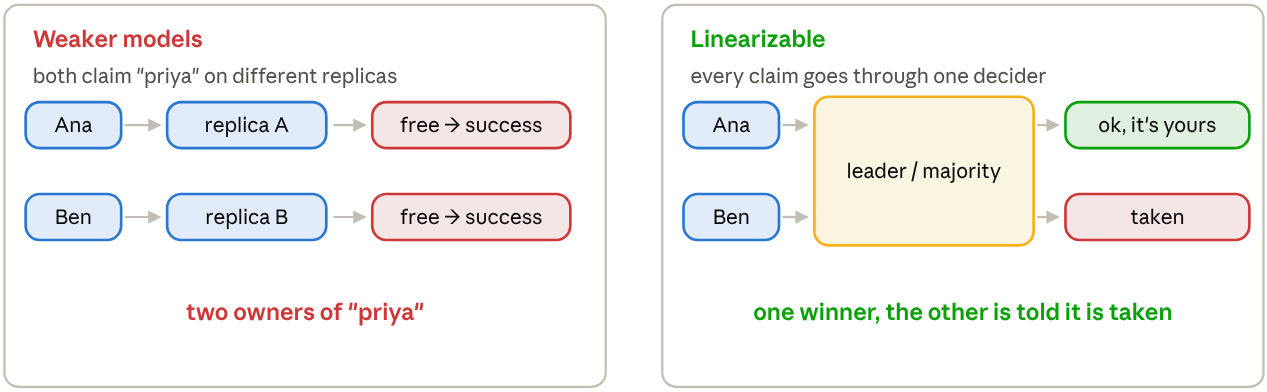
Example: two people try to claim the username "priya" from different places.

- **Weaker models:** Ana hits replica A, Ben hits replica B. Both see the name as free, both are told success. **Two owners of "priya".**
- **Linearizable check:** the claim goes through the leader (or a majority). One wins, and the other is told the name is taken.

The same applies to the **last pair of shoes in stock** and to **moving money**: one decider, never two.

The cost is coordination. Every such operation must go through the leader, or a majority, before answering. That adds latency, and if the network splits, some side has to stop answering.

Linearizable: one decider, one winner for the username



The cost is coordination: every such operation goes through the leader, or a majority, before answering. That adds latency, and if the network splits, some side has to stop answering.

claiming a username: weaker vs linearizable

The ladder, in practice

Four rungs, from cheapest to strongest:

Rung	Guarantee	Use it for
Eventual	If writes stop, all copies agree	Counts, feeds, anything that can be briefly stale
Read your writes, monotonic	You see your own updates, and nobody sees time go backwards	Showing a profile (the writer, then everyone else)
Causal	If B depends on A, everyone sees A first	Comments, replies, threads
Linearizable	One copy, every operation at a single instant	Claiming a username, moving money, the last item in stock

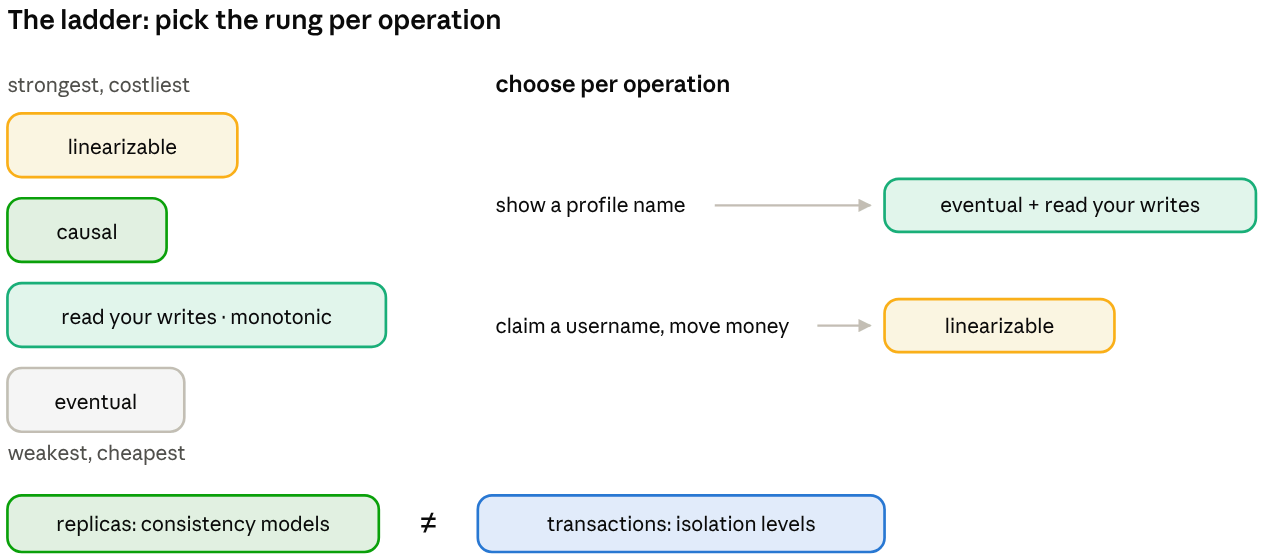
Choose per operation, not per system.

- Show a profile name: **eventual + read your writes.**
- Claim a username, move money: **linearizable.**

One thing not to mix up: replica consistency models are **not** transaction isolation levels. Consistency models are about copies of the same data on different replicas. Isolation levels are about concurrent transactions on the same data (see the Database Isolation

notes).

Source noted on the slide: AWS DynamoDB docs and AWS News Blog, 2 Dec 2020.

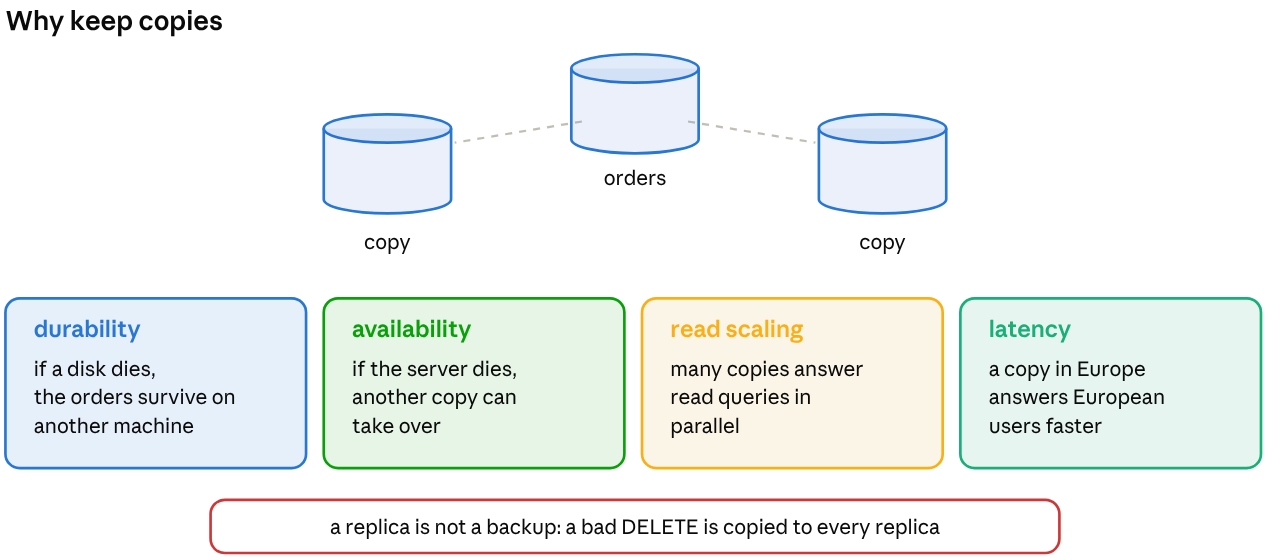


the ladder and the choice per operation

Replication: why keep copies

Everything above (eventual, read your writes, monotonic) exists because of **replication**: keeping the same data on more than one machine. Using the `orders` data as the example, there are four reasons to keep copies:

- **Durability:** if a disk dies, the orders survive on another machine.
- **Availability:** if the server dies, another copy can take over.
- **Read scaling:** many copies answer read queries in parallel.
- **Latency:** a copy in Europe answers European users faster.



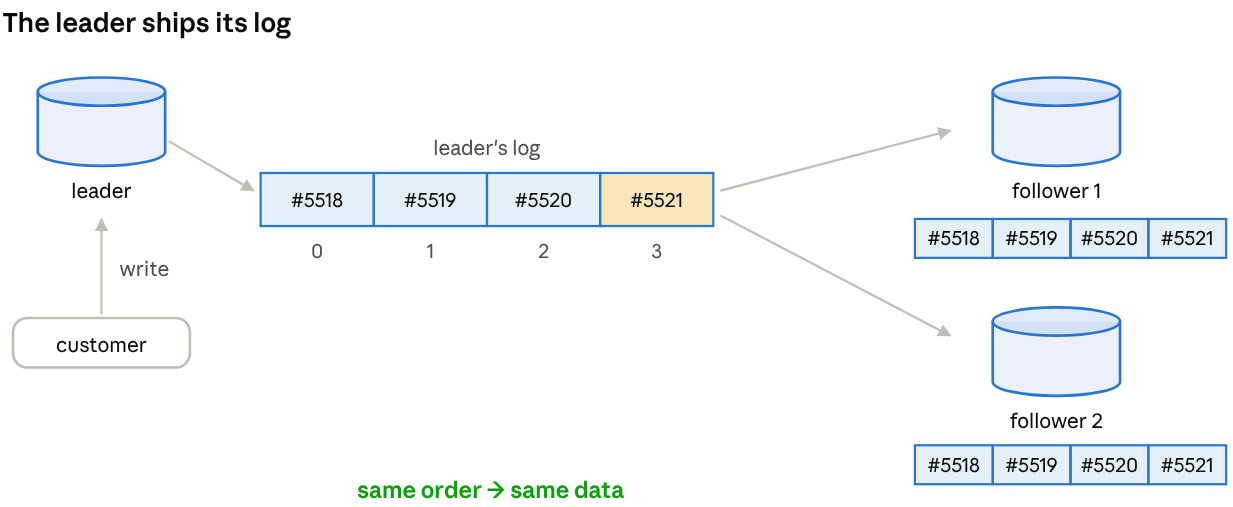
why keep copies

One thing replication is not: a backup. If someone runs `DELETE FROM orders;`, every replica faithfully deletes too. A backup is a separate, point-in-time copy. In SQL Server terms that means your full, differential and log backups for point-in-time restore, not the availability group replicas (my note).

The leader ships its log

How do followers stay in sync? The leader writes every change to an **ordered log**: #5518, #5519, #5520, #5521 (positions 0 to 3). Each follower **replays the entries in exactly the same order**, so it ends up with exactly the same data.

Same order, same data. This also explains replication lag: a follower's lag is just how far along the log it has got. That log position is the version number from the read your writes section (the LSN in PostgreSQL and SQL Server).



the leader ships its log

Synchronous or asynchronous

The question: **when is the order "saved"?**

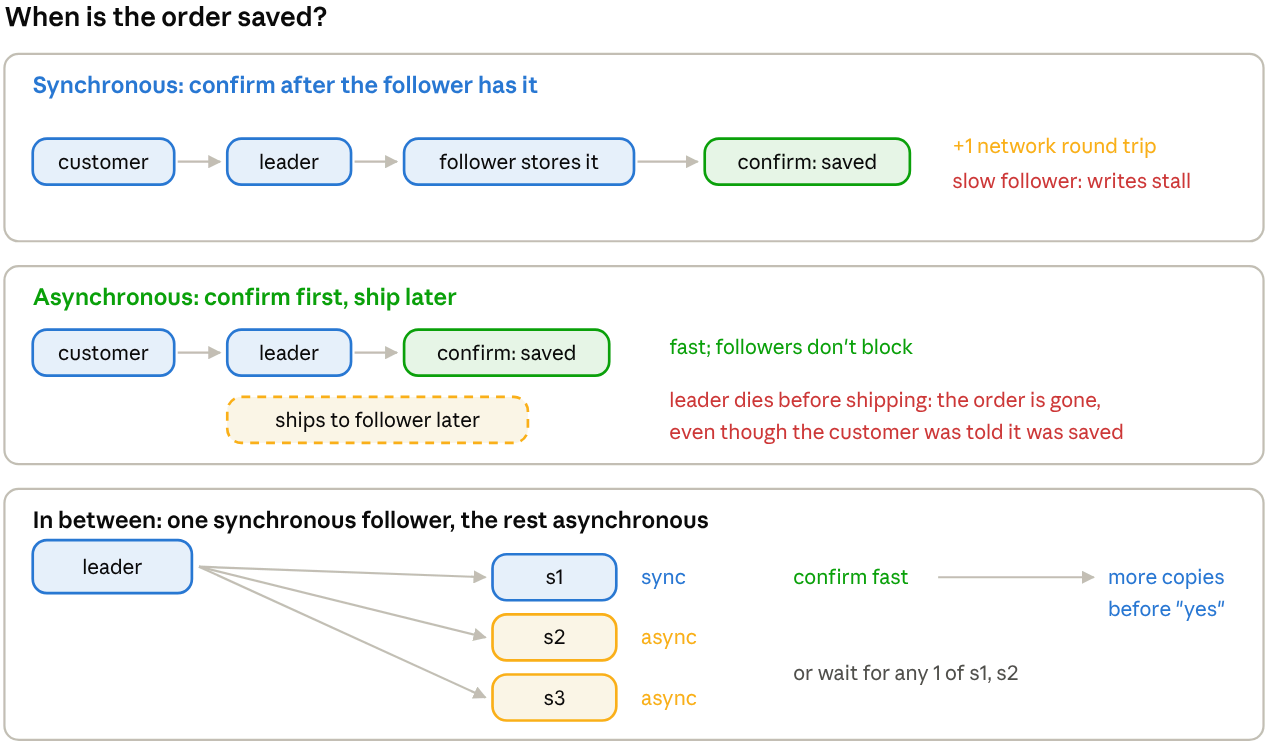
- **Synchronous:** the leader waits for the follower to confirm before telling the customer "saved". The order survives if the leader dies. The cost is **+1 network round trip** on every write, and a **slow follower makes writes stall**.
- **Asynchronous:** the leader confirms right away and ships the entry afterwards. Writes are fast and a dead or slow follower blocks nothing. But if the leader dies before shipping the entry, **the order is gone, even though the customer was told it was saved.**
- **In between** (most real setups): make **one follower synchronous and the rest asynchronous**, or wait for **any one of several followers**.

PostgreSQL example (from the slide):

```
synchronous_standby_names = 'ANY 1 (s1, s2)'
synchronous_commit = on
```

A commit now waits until any one of s1 or s2 has the entry. s3 stays asynchronous. That is the trade: every millisecond you wait buys a little more durability, from "confirm fast" to "more copies before yes".

In SQL Server, availability group replicas run in synchronous-commit or asynchronous-commit mode (my note, check your version).



synchronous, asynchronous, in between

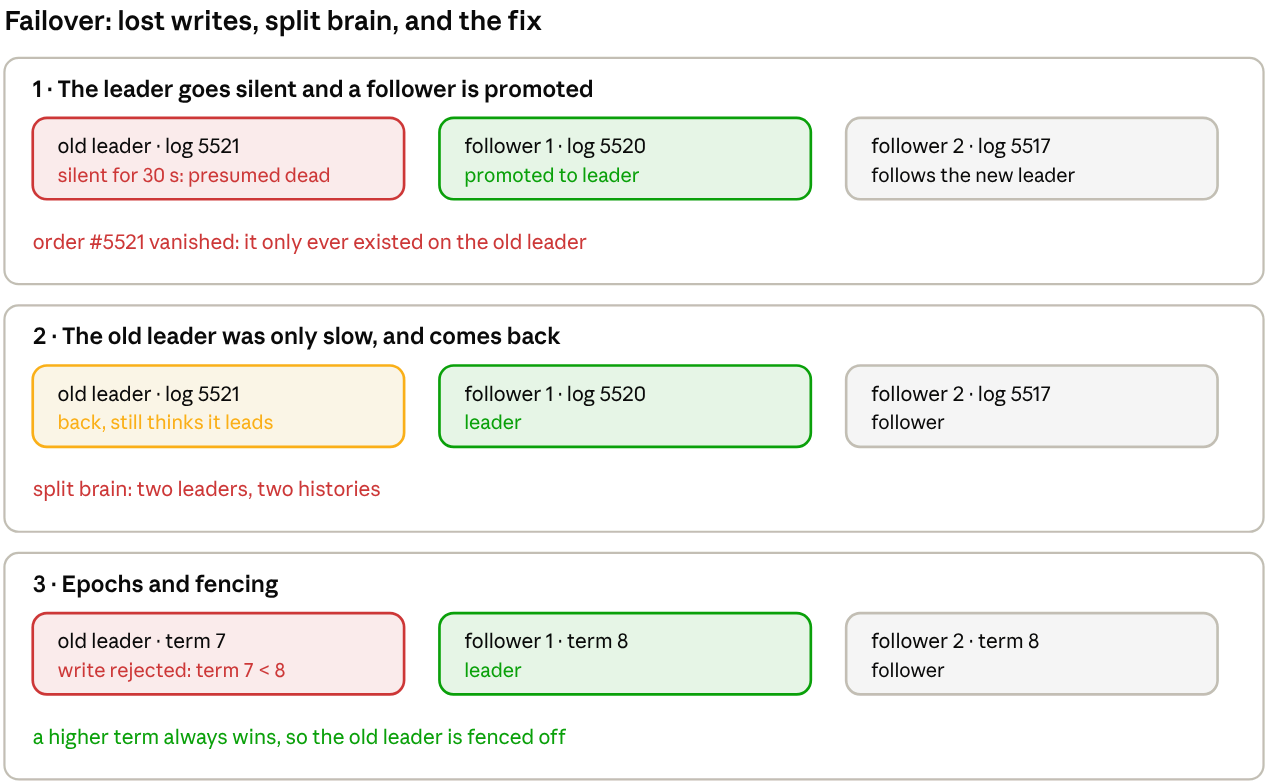
Failover

When the leader dies, a follower is promoted. Two things can go wrong.

1. Lost writes. The leader is declared dead after no reply for 30 s. With asynchronous replication the new leader may be missing the last few writes. The old leader had log 5521, follower 1 had 5520 and follower 2 had 5517. Promote follower 1 (the most up to date), and **order #5521 simply vanishes.** The 30 s is a trade-off: too short gives false failovers, too long gives a longer outage (my note).

2. Split brain. The old leader might not be dead. It was just slow, or cut off. It comes back **still believing it is the leader and accepts writes.** Now there are **two leaders and two diverging histories.** That is roughly what hit GitHub: writes on both coasts that neither side had.

The guard: epoch numbers (terms) and fencing. Every new leader gets a **higher term**, and writes from a lower term are rejected. The old leader is on term 7, the new one on term 8, so the old leader's write is **rejected: term 7 < 8.**



failover, split brain, fencing

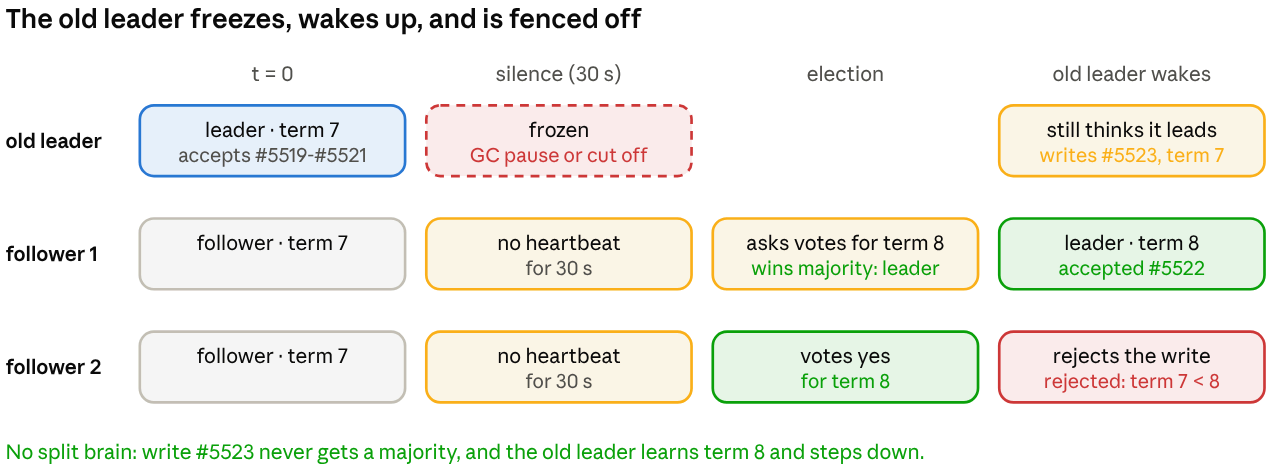
Epochs and fencing, in detail

The problem in one sentence: a node cannot tell, from the inside, that it has been replaced. A leader that was frozen for 40 s wakes up with no memory of the gap and carries on as if nothing happened. So the *other* side has to refuse its writes.

A worked example with the `orders` database and three nodes:

Step	Old leader (term 7)	Follower 1	Follower 2	Result
1	Leads on term 7, accepts #5519 to #5521	Follows	Follows	Normal
2	Freezes (a long GC pause, or the network is cut)	No heartbeat	No heartbeat	Silence
3	Still frozen	After 30 s, bumps its term to 8 and asks for votes	Votes yes for term 8	Follower 1 has 2 of 3 votes, so it becomes leader on term 8

Step	Old leader (term 7)	Follower 1	Follower 2	Result
4	Still frozen	Accepts order #5522 on term 8	Copies it	The cluster carries on
5	Wakes up, still believes it leads, sends #5523 on term 7		Replies "my term is 8"	**Rejected: term 7 < 8.** The old leader steps down



epochs and fencing: the timeline

Without the term check, the old leader would accept #5523 locally and tell the customer "saved". Two histories would now exist, and one of them has an order nobody else knows about. With the check, #5523 is never committed and the client gets an error instead.

The rules, in plain words

1. Every node stores a **term** (the epoch): a number that only goes up. It is written to disk, not kept only in memory.
2. To become leader, a node **adds 1 to its term and asks for votes.** Each node votes **at most once per term**, and only for a candidate whose log is at least as up to date as its own. A **majority** of votes wins. (This is also why the most up-to-date follower gets promoted.)
3. **Every message from a leader carries its term.**
4. A node that sees a **higher term** than its own adopts it and **steps down** to follower. A node that receives a message with a **lower term rejects it** and replies with its own term.
5. A leader **commits a write only after a majority accepts it.**

Why a majority makes it safe. Any two majorities of the same cluster share at least one node. Once the new leader has won term 8 with a majority, at least one node in *any* majority the old leader tries to reach already knows about term 8, so it refuses. The old leader can never collect a majority of acknowledgements. This is the same overlap idea as W + R > N.

Pseudocode

```
class Node:
    def __init__(self):
        self.term = 0               # persisted to disk
        self.voted_for = None       # persisted to disk, per term
        self.role = "follower"
        self.log = []

    def step_down(self, new_term):
        self.term = new_term
        self.voted_for = None
        self.role = "follower"

    # 1. No heartbeat for a while: try to become leader
    def start_election(self):
        self.term += 1
        self.role = "candidate"
        self.voted_for = self.id
        votes = 1                                   # my own vote
        for peer in self.peers:
            r = peer.request_vote(self.term, self.id, self.last_log_pos())
            if r.term > self.term:                  # someone is already
ahead
                self.step_down(r.term)
                return
            if r.granted:
                votes += 1
        if votes > (len(self.peers) + 1) // 2:      # majority
            self.role = "leader"
            self.send_heartbeats()

    # 2. Another node asks for my vote
    def request_vote(self, term, candidate, candidate_log_pos):
        if term < self.term:
            return Reply(self.term, granted=False)
        if term > self.term:
```

```
            self.step_down(term)
        ok = (self.voted_for in (None, candidate)
              and candidate_log_pos >= self.last_log_pos())
        if ok:
            self.voted_for = candidate              # persist before replying
        return Reply(self.term, granted=ok)

    # 3. A leader sends me a write: this is the fencing check
    def append(self, term, entry):
        if term < self.term:
            return Reply(self.term, ok=False)       # stale leader: fenced
off
        if term > self.term:
            self.step_down(term)
        self.log.append(entry)
        return Reply(self.term, ok=True)

    # 4. I am the leader and a client sends a write
    def write(self, entry):
        acks = 1
        for f in self.followers:
            r = f.append(self.term, entry)
            if r.term > self.term:                  # I have been replaced
                self.step_down(r.term)
                raise NotLeader()
            if r.ok:
                acks += 1
        if acks > (len(self.followers) + 1) // 2:
            self.commit(entry)
        else:
            raise WriteFailed()
```

Fencing tokens for things outside the cluster

Terms protect the replicas from each other. The same trick protects any **shared resource** (a storage service, a database table, a file) from a worker that *thinks* it still holds a lock.

Example: worker A takes a lease on the refund batch and pauses. The lease expires, worker B takes it, and then A wakes up and writes anyway.

1. A acquires the lease and receives **token 33**. A pauses for 40 s.
2. The lease (30 s) expires. B acquires it and receives **token 34**, and writes.

3. A wakes up, still believing it holds the lease, and writes with token 33.
4. The resource has already seen 34, so it **rejects 33.**

A timer in A's own head is not enough, because A cannot know it was paused. **The resource must do the check.**

```
# Lock service: every grant gets a bigger number
def acquire(resource):
    token = counter.increment(resource)     # 33, 34, 35 ...
    return Lease(token, expires_in=30)

# The protected resource enforces the fence
def write(resource, data, token):
    if token < last_token[resource]:
        raise Rejected("stale token")        # fenced off
    last_token[resource] = token
    apply(data)
```

In SQL Server, do the check and the real write in **one transaction**:

```
CREATE TABLE fence (
    resource   varchar(50) PRIMARY KEY,
    last_token bigint NOT NULL
);

BEGIN TRAN;

UPDATE fence SET last_token = @token
WHERE resource = 'refund-batch' AND last_token <= @token;

IF @@ROWCOUNT = 0
BEGIN
    ROLLBACK;
    THROW 50001, 'stale token: fenced off', 1;
END

-- the real write
UPDATE refunds SET status = 'done' WHERE batch_id = @batch;

COMMIT;
```

It is the same pattern as the version check in lost update fix 3: a conditional UPDATE that only succeeds for the newest holder.

Practical gotchas

- **Persist the term and the vote before replying.** If a node forgets who it voted for after a restart, it can vote twice in one term.
- **Lease timers are a hint, not a guarantee.** Pauses and clock drift mean a node can outlive its lease. Only the term or token check is reliable.
- **Fencing only works if the resource enforces it.** A token that nobody checks protects nothing.
- **Use real implementations** (below) instead of writing your own election code.

From my own knowledge, not your slides: Raft calls this number the **term**, Kafka calls it the **leader epoch**, and ZooKeeper carries an epoch inside its transaction id (zxid). etcd and Consul are built on Raft.

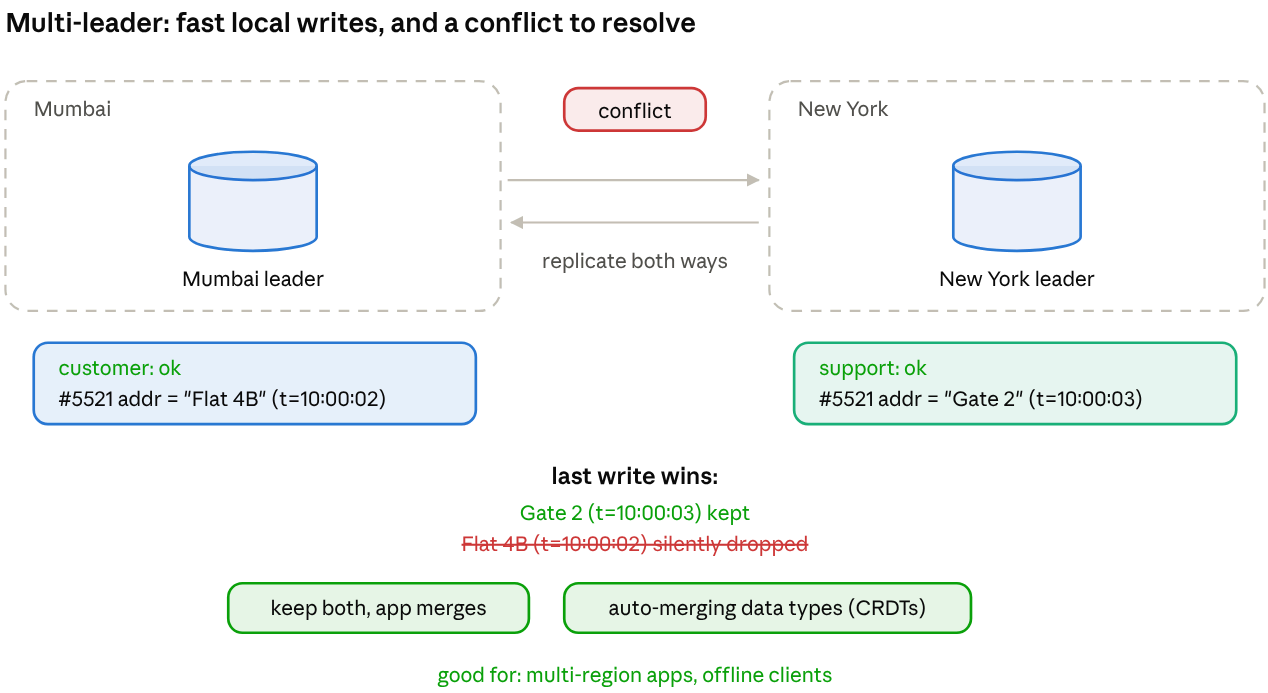
Multi-leader

A single leader is also a **single place every write must travel to**. For a shop with customers in Mumbai and New York, that is a long trip. So some systems run **multiple leaders, one per region.** Each accepts writes locally and replicates to the other. Writes are fast everywhere. It is a good fit for multi-region apps and offline clients.

The catch: **two leaders can change the same row at the same time.**

Example: a customer updates the delivery address on order 5521 in Mumbai ("Flat 4B", t=10:00:02), while support changes it in New York ("Gate 2", t=10:00:03). Both get "ok". When the leaders replicate, they conflict. Ways to resolve it:

1. **Last write wins:** keep the later timestamp and silently drop the other. Here "Gate 2" is kept and "Flat 4B" is dropped. Simple, but **it loses data, and clocks can lie.**
2. **Keep both versions** and ask the application to merge them.
3. **Auto-merging data types (CRDTs)**, designed to merge on their own.



multi-leader and conflicts

Leaderless quorums

The third style has **no leader at all.** This is the Dynamo (2007) design, used by Cassandra and Riak.

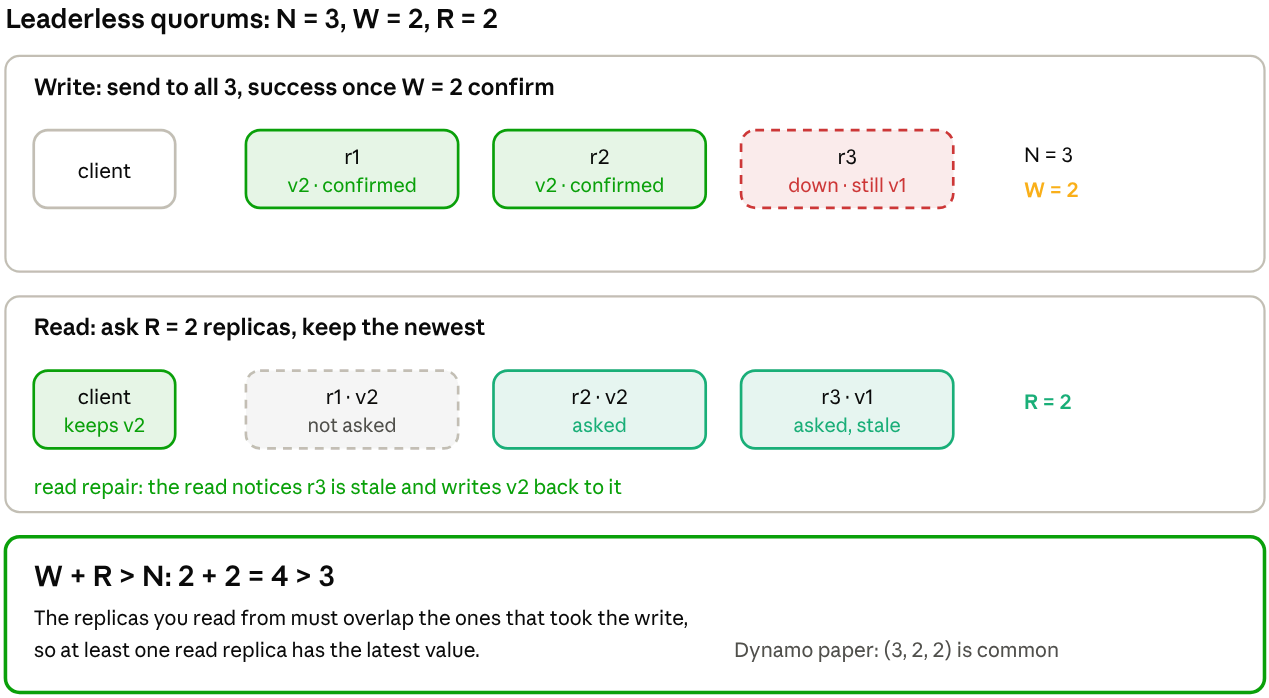
- Every key lives on **N replicas**. Say N = 3: r1, r2, r3.
- **Write:** the client sends the write to all three and treats it as successful once **W of them confirm**. Say W = 2.
- If r3 is down, r1 and r2 take v2 and r3 stays on v1. The write still succeeds, because 2 confirmed.
- **Read:** the client reads from **R replicas** (R = 2), compares the versions and **keeps the newest**. If it reads r2 (v2) and r3 (v1), it keeps v2.

The trick: W + R > N. The replicas you read from must overlap with the ones that took the write. Here 2 + 2 = 4, which is more than 3, so **at least one read replica has the latest value.** The Dynamo paper reports (N, W, R) = (3, 2, 2) as a common setting. With that setting any one replica can be down and both reads and writes still work (my arithmetic).

Stale replicas get fixed along the way:

- **Read repair:** a read that notices an old copy writes the new one back.

- **Background processes** compare replicas and repair the differences (often called anti-entropy).



leaderless quorums: N, W, R

In plain words: what if one replica missed the write?

Setup: N = 3 and W = 2. The write **v2** reached **R1 and R3**, and 2 confirmations means the write succeeded. **R2 was down**, so it still holds **v1**. Now R2 is back, and reads arrive.

Question 1: a read hits R2, which says v1. How does anyone know v1 is old?

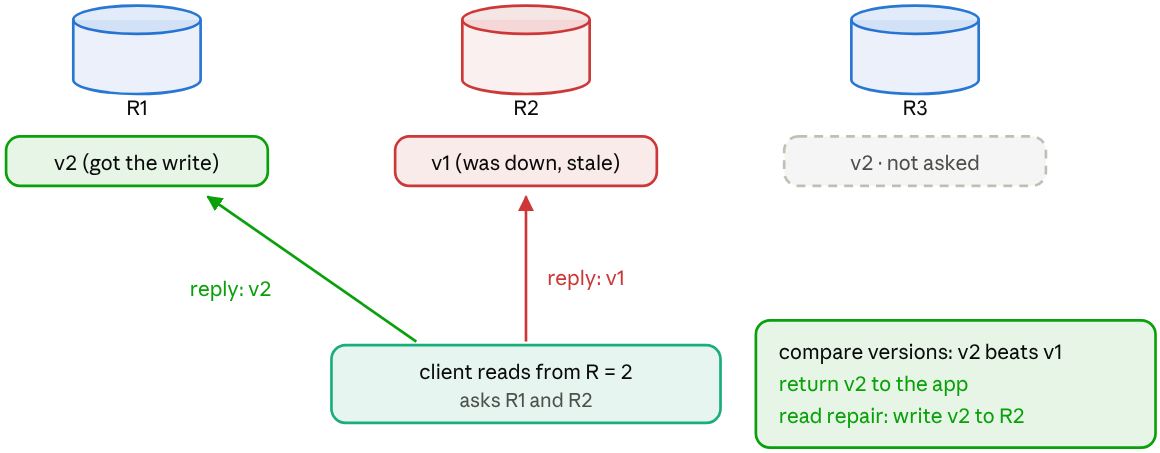
R2 doesn't know, and it doesn't need to. The reader never trusts a single replica. With R = 2 it asks **two replicas**, and each one answers with its **value and its version**. The client then simply compares the versions: v2 is bigger than v1, so v1 is old, and it keeps v2.

Here is every pair of replicas a read could ask:

Read asks	Answers	Client returns
R1 and R2	v2 and v1	**v2** (R2 gets repaired)
R2 and R3	v1 and v2	**v2** (R2 gets repaired)
R1 and R3	v2 and v2	**v2**

There is no pair that returns only R2's v1, because a read needs **two** answers and any two replicas include R1 or R3. That is the W + R > N guarantee in action: the write set {R1, R3} and any read set of two must overlap.

R2 missed the write: the reader spots it by comparing versions



R2 does not need to know it is stale: the reader finds out by comparing, then repairs R2.

R2 is stale: compare versions and repair

- **If R2 is still down during the read:** the read just goes to R1 and R3 and gets v2 twice.
- **How R2 catches up:** when a read notices R2 is stale, it writes v2 back to R2 (**read repair**). If nobody ever reads that key, the background repair fixes it. In Dynamo-style systems another node can also hold the write as a "hint" and hand it to R2 when it returns (my note).
- **What goes wrong with R = 1:** if the read asked only R2, it would return v1. 2 + 1 = 3 is not greater than 3, so the overlap guarantee is gone. That is why R must be 2 here.

Question 2: what if an update and a read happen at the same time?

They don't wait for each other. The write is sent to all three replicas, the read is sent to two of them, and **each replica answers with whatever it holds at that instant.** The **version number** decides which answer wins. The coordinator collects the replies, picks the highest version, returns it, and quietly sends the newest value to any replica that was behind.

Example: a new write **v3** is in flight. R1 has already applied it, but R2 (still v1) and R3 (v2) have not yet.

Read asks	Answers	Client returns
R1 and R2	v3 and v1	**v3**
R1 and R3	v3 and v2	**v3**
R2 and R3	v1 and v2	**v2**

Both v3 and v2 are acceptable here, because the write is **not finished yet**: only 1 of the 2 required confirmations has arrived. Once W = 2 replicas have confirmed v3, the write is done, and **every read after that overlaps at least one replica holding v3**, so it returns v3.

One caution from my own knowledge: during that in-flight window, two readers can see different answers, and one reader can even see v3 and then v2. Read repair spreads v3 so it settles quickly, but this is exactly why quorums alone are not linearizable (see the cautions below).

Two cautions, both from my own knowledge and not your slides:

- **How is "newest" decided?** By the version. Dynamo used vector clocks, while Cassandra compares timestamps and lets the last write win, so the multi-leader conflict problem comes back here.
- **W + R > N is not linearizable.** Sloppy quorums, concurrent writes, and a write that reached fewer than W replicas before failing can all still give stale or odd reads. For claiming a username or moving money, use one decider (the leader or a majority). In Cassandra, reading and writing at QUORUM gives W + R > N. Also, the Amazon DynamoDB service is not the same design as the Dynamo paper.

Single leader, multi-leader, leaderless

Style	Good for	The cost	Examples
Single leader	Simple, one order of writes	One machine takes every write	PostgreSQL, MySQL
Multi-leader	Fast local writes, multi-region and offline	Conflicts	
Leaderless	Rides out slow or dead nodes	Versions, repair, weaker order	Cassandra, Riak

How replication connects to the ladder

- Asynchronous followers lag, and that lag is **why reads are eventual.** Read your writes and monotonic reads are the fixes.
- Synchronous replication buys **durability.** Linearizable needs a **single decider**, the leader or a majority.
- Multi-leader **gives up the single decider**, so it needs conflict resolution. It is the wrong tool for something like claiming a username (my note).

Replication in real databases

From my own knowledge, not your slides, so please check against your versions.

System	What to know
PostgreSQL	Streaming replication is asynchronous by default. `synchronous_standby_names` and `synchronous_commit` make a commit wait for standbys.
SQL Server / Azure SQL	Availability groups have synchronous-commit and asynchronous-commit replicas. Automatic failover needs synchronous commit, and the cluster quorum decides who leads, which guards against split brain. Azure SQL geo-replication is asynchronous.
Azure Cosmos DB	Multi-region writes with conflict resolution: last writer wins by default, or a custom merge procedure.
DynamoDB	Global tables are multi-region, with last writer wins.

CAP theorem

The three letters:

- **C: consistency** here means **linearizable**: the system acts like a **single copy**. (It does *not* mean eventual.)

- **A: availability:** every request to a working node gets a **real answer**, not an error and not "try later".
- **P: partition tolerance:** the system **keeps working when messages between nodes are lost.**

The result: during a partition you cannot have both C and A.

Choice Per Operation

Example 1: the last seat (choose C)

Two primary servers, one in Mumbai and one in New York, and **one seat left**. The link between them breaks.

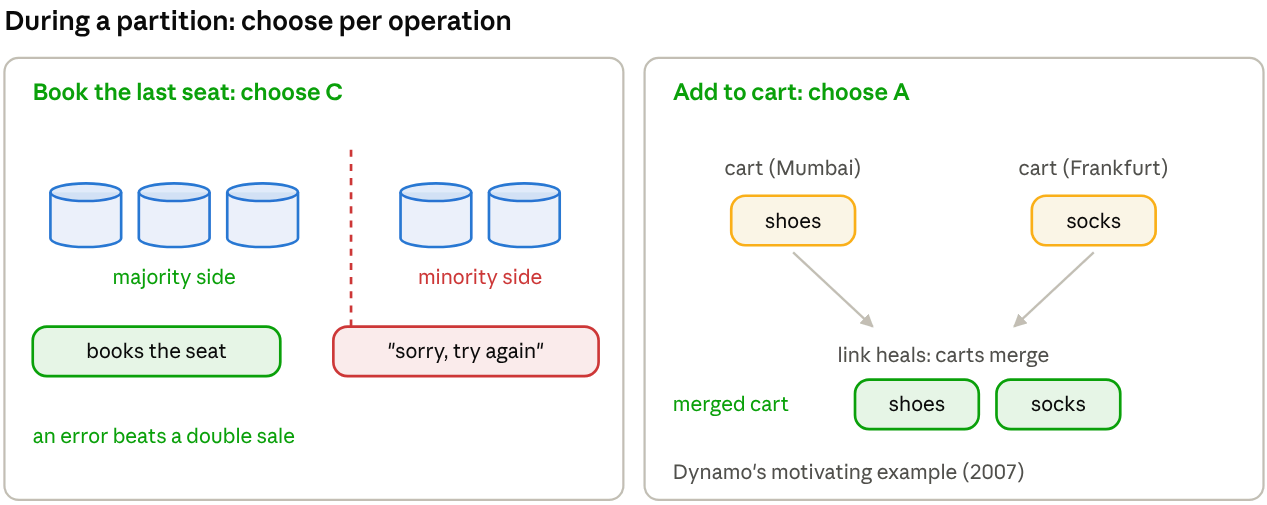
- **If we choose availability (and speed):** the Mumbai user books the seat from the nearby Mumbai data center, and a New York user books the same seat from the New York data center. One seat, **two bookings.** Wrong.
- **If we choose consistency:** one side must refuse. That user sees **"sorry, try again".**

An error beats a double sale. (In the slide there are five nodes. The three-node **majority side** keeps booking and the two-node **minority side** says sorry. With only two nodes neither side has a majority, so you decide which side keeps serving, or you refuse on both.)

Example 2: the shopping cart (choose A)

A customer adds shoes to the cart from Mumbai and socks from Frankfurt, while the link is broken. Both writes succeed. When the link heals, the two carts are **merged** into one cart with shoes and socks. Refusing the add-to-cart would lose a sale, so here we choose availability. This was **Dynamo's motivating example (2007).**

So the choice is **per operation**, not per system. It is the same idea as "choose per operation" on the ladder above.



Same partition, a different choice per operation.

CAP: choose per operation

Is the partition optional?

No. Networks do fail, so you cannot opt out of partitions. The real question is **what to give up during one.** (A "CA" distributed system is not a real option, my note.) Brewer's plan for partitions has three steps:

1. **Detect the partition.** Timeouts and missed heartbeats.
2. **Partition mode: limit what is allowed.** For example keep add-to-cart working, but refuse booking the last seat.
3. **Recover: repair the damage.** Merge the carts, or use a compensating action such as a refund (see the booking notes).

A partition is not only two data centers

Your example of calling an LLM through your own API or backend is the same shape. The request or the reply can be lost or slow, and from your side you **cannot tell whether the work happened** (like the timeout cases in the retries notes). So you decide up front: fail with an error, retry with an idempotency key, or degrade (a cached or default answer). Strictly speaking, CAP is about a replicated data store, so this is an analogy, but the design thinking is the same: detect it, limit what is allowed, recover.

PACELC: the everyday trade-off

Daniel Abadi, 2012. CAP only talks about partitions, and partitions are rare. Most of the time the network is fine, and you **still** have a trade-off. PACELC adds that second half:

If there is a Partition: choose Availability or Consistency. Else (normal operation): choose Latency or Consistency.

Why is there a trade-off when nothing is broken? Keeping copies in agreement costs time. Take the Mumbai and New York example with a healthy link, and a write that arrives in Mumbai:

- **Choose latency:** confirm to the user straight away and copy to New York in the background (asynchronous). It is fast, but New York may briefly read old data.
- **Choose consistency:** wait until New York confirms before answering (synchronous). New York is never stale, but **every write pays the round trip across the ocean.**

That is the same choice as synchronous versus asynchronous replication above, and the same choice as a small or large R and W in a quorum.

How to read the grid

The label has two halves. **The first pair is the partition choice (PA or PC). The second pair is the normal-time choice (EL or EC).** That gives four boxes:

Box	During a partition	Normally	Examples (default settings)
PA/EL	Stay available	Prefer low latency	Dynamo, Cassandra, Riak
PA/EC	Stay available	Prefer consistency	MongoDB (as of 2012)
PC/EL	Stay consistent	Prefer low latency	Yahoo PNUTS
PC/EC	Stay consistent	Prefer consistency	VoltDB / H-Store, Megastore, Bigtable / HBase

The yellow arrow in the grid means a system can **move to another box through settings.** Cassandra is PA/EL by default, but if you **raise R and W per query** (so W + R > N) it behaves more like PA/EC: you gain consistency and pay with latency.

What each box means, in plain words

Spell the label out in four short pieces:

- **PA** = "if there is a **P**artition, I choose **A**vailability" (keep answering).
- **PC** = "if there is a **P**artition, I choose **C**onsistency" (refuse rather than be wrong).
- **EL** = "**E**lse, on a normal day, I choose **L**atency" (answer fast).
- **EC** = "**E**lse, on a normal day, I choose **C**onsistency" (wait until the copies agree).

A memory trick: the **first pair** answers "what do I do when the link breaks?" and the **second pair** answers "what do I pay on a normal day?"

Take one operation, a customer **changing the delivery address** on an order in Mumbai, with a copy in New York:

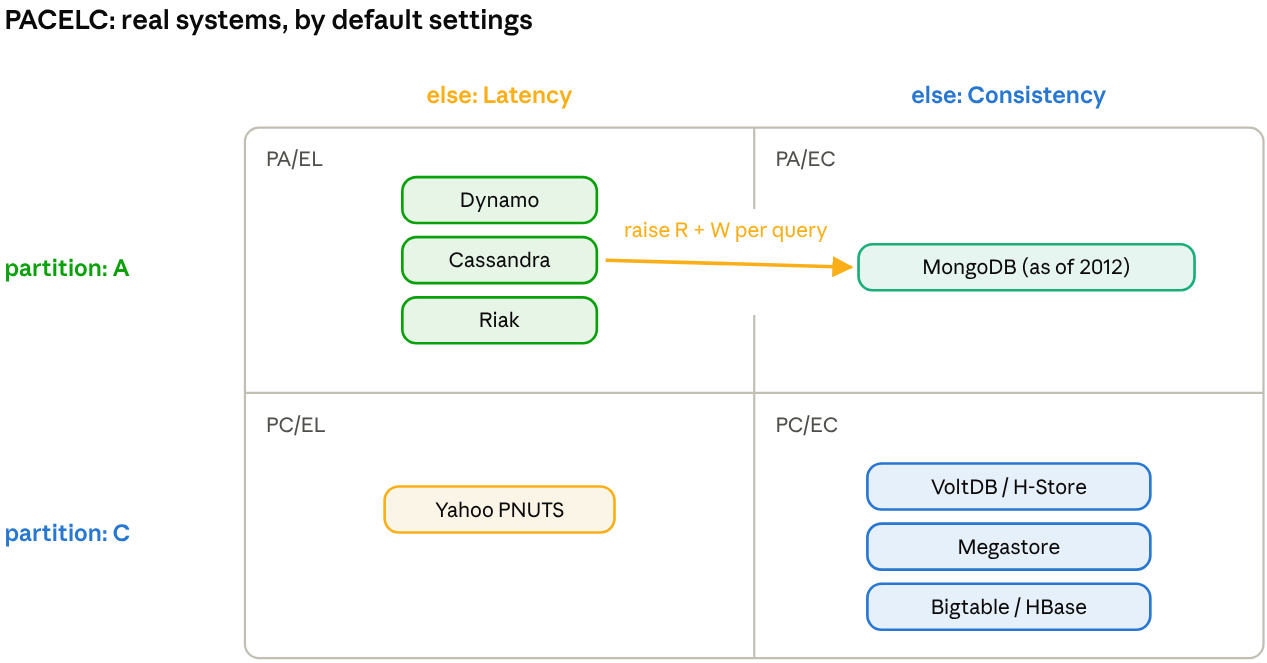
Box	In plain words	Healthy link (normal day)	Link broken (partition)
PA/EL	Always up, always fast	Mumbai confirms **instantly** and copies to New York later. New York may show the old address for a moment	**Both sides keep accepting** address changes. The differences are merged later
PA/EC	Up in a failure, careful on a normal day	Mumbai **waits until New York has it**, then confirms. Slower, never stale	Mumbai **still confirms** on its own, even though New York cannot be told
PC/EL	Strict in a failure, fast on a normal day	Mumbai confirms **instantly** and copies later	The address change is **refused** ("try again") until the link is back
PC/EC	Correctness first, always	Mumbai **waits for New York** (or a majority) before confirming	The side that cannot reach the others **refuses** ("sorry, try again")

When would you pick each?

- **PA/EL:** things where a stale or merged answer is fine and speed or uptime matters most: shopping cart, likes, view counts, a profile name.
- **PA/EC:** you want correct reads day to day, but you still prefer to keep serving during an outage and sort the differences out afterwards.

- **PC/EL:** the unusual box. Partitions are rare, so being strict then costs little, but speed on a normal day is felt every day (my reading of it).
- **PC/EC:** things where being wrong is worse than being slow or briefly unavailable: the last seat, moving money, claiming a username.

Two things to remember. First, the box describes a system's **default settings**, and settings can move it (Cassandra moves toward PA/EC when you raise R and W). Second, the choice is really **per operation**: one product can treat the cart as PA/EL and the last seat as PC/EC.



Classified by default settings: other settings can move a system to a different box.

PACELC grid

Three misconceptions

1. **CAP's C is not ACID's C.** CAP's C means the **copies agree** (linearizable). ACID's C means **your invariants hold** (rules like stock never below zero).
2. **AP does not mean always inconsistent.** An AP system is usually consistent **within milliseconds.** It only differs during a partition or while replication catches up.
3. **"CP database" is fuzzy.** The behavior varies **per query and per setting** (for example Cassandra at ONE or QUORUM, or DynamoDB with `ConsistentRead`), so the label is only a rough guide.

Where this fits in the earlier notes

Here	Earlier notes
A cache is another copy that can be stale. TTL and invalidation are how it eventually agrees	Caching
The seat map is **eventual** (a few seconds stale). Holding a seat is **linearizable** per seat: one conditional write, one winner	Ticket Booking System
Replica consistency models are not isolation levels	Database Isolation
The version token is the same idea as the version check in lost update fix 3	Same Database Isolation notes

Recap

1. **Eventual:** if writes stop, all copies agree.
2. **Read your writes:** the writer sees their own update. Use a 60 s leader window or a version token.
3. **Monotonic reads:** pin each reader to one replica so time never goes backwards.
4. **Causal:** the write carries its dependency, and replicas hold it back until the parent arrives.
5. **Linearizable:** one decider (leader or majority) for usernames and money. It costs latency and availability.
6. Choose per operation. Consistency models are not isolation levels.

Replication

- Copies give durability, availability, read scaling and lower latency. A replica is not a backup.
- The leader ships an ordered log and followers replay it in the same order, so same order gives same data.
- Synchronous is safer but slower, and a slow follower stalls writes. Asynchronous is fast but can lose the last writes on failover. Most setups use one synchronous follower or "any 1 of N".
- Failover can lose writes and cause split brain. Guard it with epochs (terms) and fencing.

- Multi-leader gives fast local writes but needs conflict resolution. Last write wins loses data, so keep both and merge, or use CRDTs.
- Leaderless quorums: every key lives on N replicas, a write succeeds once W confirm, and a read asks R and keeps the newest. If W + R > N the read set overlaps the write set. Read repair and background repair fix stale replicas. Quorums alone are not linearizable.

CAP and PACELC

- CAP: C means linearizable, A means every working node gives a real answer, P means it works when messages are lost. During a partition you cannot have both C and A.
- Partitions are not optional. Detect them, limit what is allowed, then recover. Choose per operation: an error beats a double sale, but a cart can merge later.
- PACELC: if Partition, choose A or C. Else, choose Latency or C. The everyday cost of consistency is waiting for the other copies.
- Misconceptions: CAP's C is not ACID's C, AP is not always inconsistent, and "CP database" is fuzzy.